# HLD Level 2.2

**Design a Thread Pool / Task Execution Service — Complete Interview Question & Answer**

**Interview question: Design a task execution service that accepts tasks from multiple producers and executes them concurrently using a bounded number of reusable worker threads. Explain queueing, synchronization, overload handling, failures, results, cancellation and graceful shutdown.**

## 1. Requirements

**Functional:** accept tasks from multiple producers; queue pending work; execute tasks concurrently; expose status/results where needed; handle task failures; support graceful shutdown. Extensions can include priority, cancellation, retries and deadlines.

**Non-functional:** thread safety, bounded memory, high throughput, low queueing latency, no busy waiting, predictable resource usage and failure isolation.

## 2. Why a Thread Pool?

Creating a new OS thread for every task consumes stack/kernel resources and can cause excessive context switching. A pool creates a bounded set of workers once and reuses them.

```
Producers -> Bounded Task Queue -> Worker 1
-> Worker 2
-> ...
-> Worker N
```

## 3. Main Components

**The three core components are TASK, BOUNDED QUEUE and WORKER POOL.**

A task may contain task_id, executable work, priority, state, retry_count and deadline. A simple C++ representation can use std::function<void()>, while more performance-sensitive designs may use alternatives to reduce type-erasure/allocation overhead.

## 4. Task Lifecycle

```
SUBMITTED -> QUEUED -> RUNNING -> SUCCESS
\-> FAILED

Optional: FAILED -> RETRY_WAIT -> QUEUED
Optional: CANCELLED / TIMED_OUT
```

## 5. Producer-Consumer Queue

Multiple producers push work while multiple workers pop work, so shared queue state must be synchronized. A basic C++ implementation can use std::queue, std::mutex and std::condition_variable.

## 6. Why condition_variable?

Workers should sleep when no work exists rather than repeatedly polling the queue. Producers notify a sleeping worker after adding work.

```
Queue empty -> worker waits
Producer pushes task -> notify_one()
Worker wakes -> pops task -> executes
```

## 7. Worker Algorithm

```
lock queue mutex
wait until stopping OR queue not empty
if stopping AND queue empty -> exit
move/pop one task
unlock mutex
execute task outside the lock
repeat
```

**Critical interview point: never hold the queue mutex while executing arbitrary task work. Protect the queue operation, release the lock, then run the task.**

## 8. Why the Queue Must Be Bounded

```
Producer rate = 10,000 tasks/sec
Worker capacity = 8,000 tasks/sec
Backlog growth = 2,000 tasks/sec
After 60 sec = 120,000 waiting tasks
```

An unbounded queue converts sustained overload into memory exhaustion and huge stale latency. A bounded queue creates a known resource limit.

## 9. Queue-Full Policy / Backpressure

When the queue reaches capacity, choose an explicit policy based on requirements: reject, block producer, wait with timeout, drop, drop oldest, or spill durable work to an external broker/storage layer.

**For a generic executor, rejection or bounded waiting is often a good starting policy because overload becomes visible to the producer instead of silently consuming memory.**

## 10. Worker Count

**CPU-bound tasks:** worker count is usually near available CPU cores; far more runnable threads may only increase context switching.
**I/O-bound tasks:** more concurrency can help because workers spend time waiting, but the number must still be measured and bounded.

## 11. Capacity Example

```
8 workers
10 ms CPU/task
~100 tasks/sec/worker
~800 tasks/sec theoretical pool capacity

If arrivals = 1200/sec, backlog grows.
```

**Queueing cannot fix permanently insufficient processing capacity.**

## 12. Task Failure Isolation

A task throwing an exception should not unexpectedly terminate the worker. Catch task-level failures at the execution boundary, record the failure and keep the worker alive.

```
try { task(); }
catch (...) { record_failure(); }
```

## 13. Retries

Retry only transient failures and use bounded attempts with exponential backoff and jitter. Do not occupy a worker by sleeping for long retry delays; use a retry scheduler/timer that re-enqueues the task when eligible.

```
Failed task -> Retry Scheduler -> delay/backoff -> Task Queue
```

## 14. Poison Tasks

A permanently failing task must not retry forever. Enforce max attempts; then mark FAILED or send the task to a dead-letter path in a distributed executor.

## 15. Priority Scheduling

Priorities can be supported using a priority queue or separate queues, but continuous high-priority traffic can starve low-priority work. Mitigate with aging, weighted scheduling or quotas.

## 16. Cancellation

Queued work can usually be marked/removed or skipped. Running C++ work should use cooperative cancellation rather than forcibly terminating the thread. The task periodically checks a cancellation/stop request and exits safely.

## 17. Results

When callers need results, a task can complete a promise and callers can hold a future. C++ facilities include std::future, std::promise and std::packaged_task.

```
Producer -> Task -> Queue -> Worker -> promise -> future -> result
```

## 18. Graceful Shutdown

```
shutdown requested
-> stop accepting new tasks
-> wake sleeping workers
-> drain queued work OR cancel by policy
-> workers exit
-> join workers
-> release resources
```

Define shutdown semantics explicitly. Drain mode completes accepted tasks; immediate/cancel mode abandons or cancels queued tasks according to the contract.

## 19. Simplified C++ Class Shape

```
class ThreadPool {
vector<thread> workers;
queue<function<void()>> tasks;
mutex m;
condition_variable cv;
bool stopping = false;
public:
ThreadPool(size_t n);
void submit(function<void()> task);
~ThreadPool();
};
```

## 20. notify_one vs notify_all

For a normal single task submission, notify_one() usually wakes one worker. During shutdown, notify_all() is useful so every sleeping worker wakes and observes the stop condition. Calling notify_all for every single task can cause unnecessary wakeups (thundering herd).

## 21. Lock Contention

A single central queue/mutex can become a bottleneck at very high throughput. Measure first. Potential improvements include batching submissions, sharded queues, per-worker queues, shorter critical sections and work stealing.

## 22. Work Stealing

```
Worker1 -> Q1
Worker2 -> Q2
Worker3 -> Q3

If Q1 empty, Worker1 can steal work from another worker queue.
```

Work stealing can reduce central-lock contention and improve load balancing, but it is an advanced optimization rather than the first design.

## 23. Observability

Monitor tasks submitted/completed/failed per second, queue depth, queue wait time, execution time, end-to-end P50/P95/P99 latency, active/idle workers, rejection rate, retry count, CPU and memory.

> **If execution takes 10 ms but queue wait is 500 ms, the dominant problem is overload/queueing, not the task's 10 ms implementation.**

## 24. Failure Investigation

If queue depth rises continuously, inspect producer rate, completion rate, CPU, worker utilization, execution latency, downstream dependency latency and lock contention before simply adding threads.

If CPU is near 100% for CPU-bound work, more threads may worsen context switching. If CPU is low because workers wait on network I/O, additional concurrency or asynchronous I/O may help.

## 25. Complete Architecture

```
PRODUCERS
/ | \
v v v
TASK EXECUTOR
|
BOUNDED QUEUE
|
condition_variable
|
+---------+---------+
v v v
Worker1 Worker2 WorkerN
| | |
+---------+---------+
|
SUCCESS / FAILURE
|
Retry Scheduler
```

## 26. Interview Follow-Ups

**Why not one thread per task?** Unbounded thread/resource growth and context switching.

**Why bounded queue?** Prevent unbounded memory/backlog and expose overload.

**Why condition_variable?** Sleep when idle instead of busy waiting.

**Why execute outside the lock?** Keep the queue critical section short and preserve concurrency.

**How many workers?** Depends on CPU vs I/O behavior and measurement.

**What happens on full queue?** Explicit backpressure/rejection/wait/spill policy.

**How do retries work?** Bounded attempts with backoff via scheduler, not long worker sleeps.

**How do you cancel running work?** Cooperative cancellation.

**How do you shut down?** Stop intake, wake workers, drain/cancel by policy, join.

## 27. Interview-Ready Spoken Answer

"I'd design the executor around a fixed-size worker pool and a bounded thread-safe task queue. Multiple producers submit tasks into the queue, and workers wait on a condition variable rather than busy-waiting. A worker acquires the queue mutex only long enough to wait and remove a task, then releases the lock before executing it. The queue is bounded so producer throughput cannot create unlimited memory growth; when full, the service applies an explicit backpressure policy such as rejection or bounded waiting. Worker count depends on workload: CPU-bound work generally stays near available cores, while I/O-heavy work may benefit from higher measured concurrency. Task exceptions are isolated so they don't kill workers, transient failures use bounded retries with backoff, and running work uses cooperative cancellation. I would expose queue depth, queue wait, execution latency, failures and worker utilization. For graceful shutdown, stop accepting work, wake sleeping workers, drain or cancel queued tasks according to policy, join workers and release resources with RAII."

## 28. Five Decisions to Remember

1. **Do not create unlimited threads -> FIXED/BOUNDED WORKER POOL.**
2. **Do not spin when idle -> CONDITION VARIABLE.**
3. **Do not allow unlimited backlog -> BOUNDED QUEUE + BACKPRESSURE.**
4. **Do not execute tasks while holding the queue lock -> SHORT CRITICAL SECTION.**
5. **Do not abruptly destroy active workers -> GRACEFUL SHUTDOWN.**

## 29. C++ Interview Deep-Dive Topics

Be prepared to discuss std::thread/jthread, mutex, unique_lock, condition_variable predicates, spurious wakeups, move semantics, RAII, future/promise/packaged_task, exception boundaries, stop_token/cooperative cancellation, lock contention, false assumptions about lock-free structures, worker sizing, work stealing and safe destructor behavior.

## 30. Final Revision Flow

```
Requirements
-> Task representation/state
-> Bounded queue
-> Worker pool
-> condition_variable
-> Backpressure/full policy
-> Worker sizing
-> Failures/retries
-> Results/cancellation
-> Graceful shutdown
-> Metrics
-> Contention/work stealing
```